Estrutura da Blockchain

Grupo de Estudos em Blockchain

Objetivos de Aprendizagem

- Conhecer os princípios de funcionamento da *Blockchain*
- Identificar os três pilares que garantem imutabilidade e segurança

Se eu alterar um dado em um arquivo no meu computador, por

E se esse arquivo estivesse em 10.000 computadores ao mesmo tempo?

Blocos

Blocos

> É uma estrutura de dados que agrupa informações dentro da *Blockchain*

Blocos

Características

- É a **unidade básica** de armazenamento
- Contém **transações** (ou qualquer dado registrado)
- Possui uma **referência ao bloco anterior**
- Carrega um **hash próprio** (sua "impressão digital")
- É **validado** pela rede antes de ser aceito

Partes de um Bloco

1. Cabeçalho (*Header*)
2. Corpo (*Body*)

Cabeçalho

Header

> Contém metadados para identificação do bloco e conexão entre os blocos.

> O *header* garante a segurança da cadeia

Corpo

Body

A carga útil, geralmente corresponde a lista de transações.

Contém as informações práticas registradas no bloco

Campos do Cabeçalho

Campos típicos

- **Número do bloco**
- **Timestamp** – data e hora de criação
- **Hash do bloco anterior** (*previous hash*)
- **Merkle Root** – *hash*-resumo de todas as transações
- **Nonce** – número usado na mineração (*proof of work*)
- **Difficulty / Target** – dificuldade atual da rede
- **Versão** – versão do protocolo

Cada *blockchain* possui variações dos campos que utiliza.

Exemplo

Cabeçaho do Bitcoin

> O cabeçalho do Bitcoin é fixo em 80 bytes. É isso que os mineradores hasheiam no PoW.

Campo	Tamanho	Função
nVersion	4 bytes	Versão do protocolo (sinaliza soft forks)
hashPrevBlock	32 bytes	Hash SHA-256 do bloco anterior
hashMerkleRoot	32 bytes	Raiz da árvore de Merkle das transações
nTime	4 bytes	Timestamp Unix de criação

Exemplo

Bloco Bitcoin #0 (Genesis)

CABEÇALHO (80 bytes)

```
nVersion        : 01000000  (v1)
hashPrevBlock   : 0000000000000000000000000000000000000000000000000000000000000000
                  (zero — é o primeiro bloco)
hashMerkleRoot  : 3ba3edfd7a7b12b27ac72c3e67768f61
                  7fc81bc3888a51323a9fb8aa4b1e5e4a
nTime           : 29ab5f49  (2009-01-03 18:15:05 UTC)
nBits           : ffff001d  (dificuldade 1)
nNonce          : 1dac2b7c  (2.083.236.829)
```

Exemplo

Ethereum

Campo	Tamanho	Função
parentHash	32 bytes	Hash Keccak-256 do bloco pai
ommersHash	32 bytes	Constante vazia pós-Merge
beneficiary	20 bytes	Endereço que recebe as fees
stateRoot	32 bytes	Raiz da árvore de **estado global**
transactionsRoot	32 bytes	Raiz das transações
receiptsRoot	32 bytes	Raiz dos recibos (logs)
logsBloom	256 bytes	Filtro Bloom para busca rápida

Corpo do Bloco

Transações

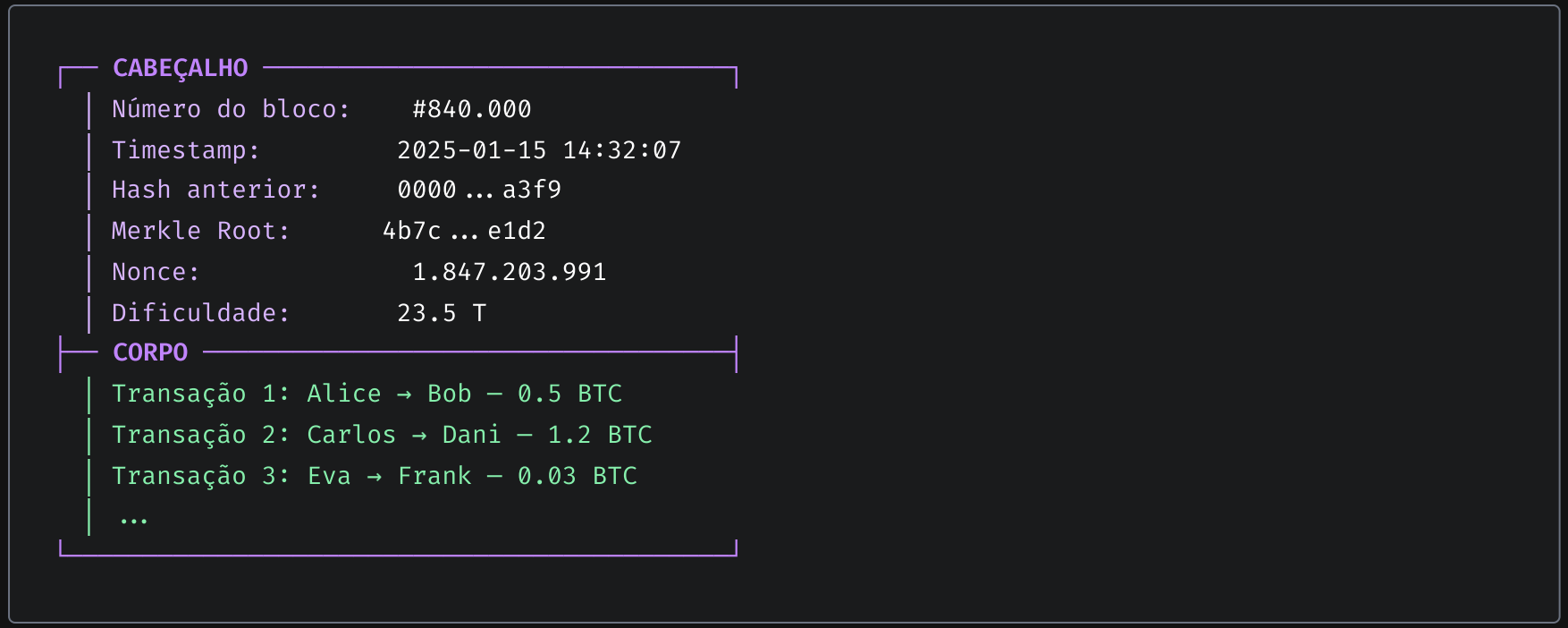
- Lista de operações registradas
- Cada transação tem remetente, destinatário e valor (ou dados)
- Cada transação é assinada criptograficamente

Árvore de Merkle

Merkle root

- Organização em **árvore binária de hashes**
- Permite verificar uma transação sem baixar o bloco inteiro
- Gera um único **Merkle Root**

Anatomia Visual de um Bloco

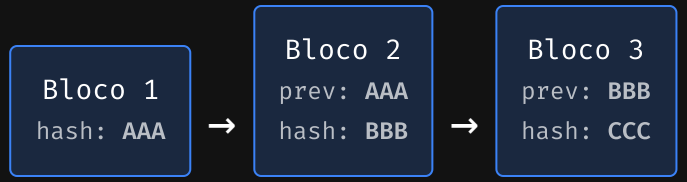


Encadeamento dos Blocos

Encadeamento

Chaining

> Cada bloco aponta para o bloco anterior através do *hash*



> O *hash* do bloco anterior é a "cola" que une tudo. É isso que forma a cadeia.

Hash

> Um *hash* é uma função que transforma qualquer entrada em uma saída de tamanho fixo.

Hash

Propriedades

- **Determinístico**; mesma entrada, mesma saída
- **Tamanho fixo**; sempre 256 bits (SHA-256)
- **Rápido**;
- **Irreversível**; não é possível desfazer a operação
- **Efeito avalanche**; 1 caractere muda tudo

Efeito Dominó

- Alterar o Bloco 1 → seu hash muda →
- Bloco 2 aponta para um hash que não existe mais →
- Bloco 3 idem → **toda a cadeia se rompe**.

> Para adulterar um bloco, seria preciso refazer todos os blocos seguintes

Imutabilidade

Visão Geral



Criptografia

Hashes encadeados



Descentralização

Cópias em milhares de nós



Consenso

Regras aceitas por todos

Os três juntos tornam a blockchain **praticamente imutável**.

🔐 Pilar 1 – Criptografia

Como funciona

- Cada bloco tem um **hash único**
- Cada bloco guarda o **hash do anterior**
- Alterar qualquer dado → hash muda
- O encadeamento de hashes forma uma "corrente inviolável"

Garantias

- **Integridade** – o dado não foi alterado
- **Autenticidade** – assinaturas digitais
- **Não-repúdio** – quem assinou, assinou

> Em resumo, a criptografia torna **detectável** qualquer tentativa de adulteração.

Pilar 2 – Descentralização

Como funciona

- A *Blockchain* é **copiada em milhares de nós**
- Cada nó valida e guarda a cópia completa
- Não existe um "servidor central"
- Se um nó cai, os outros continuam

Garantias

- **Resistência a censura** – ninguém desliga tudo
- **Alta disponibilidade** – sempre há cópias
- **Confiança distribuída** – não depende de um ator

Resumo: para alterar o passado, seria preciso convencer **a maioria da rede** – não apenas um servidor.

Pilar 3 – Consenso

Como funciona

- Regras definem **o que é um bloco válido**
- Os nós "votam" implicitamente ao aceitar/rejeitar blocos
- Só a **cadeia mais longa** (ou mais votada) vence
- Blocos inválidos são ignorados

Algoritmos comuns

- **Proof of Work (PoW)** – Bitcoin
- **Proof of Stake (PoS)** – Ethereum 2.0
- **PBFT / Raft** – redes permissionadas
- **Delegated PoS** – EOS, Tron

Resumo: o consenso garante que **todos os nós cheguem ao mesmo estado**, mesmo sem confiar uns nos outros.

Os Três Pilares em Ação

Cenário: **um atacante tenta alterar uma transação antiga**.

1. 🔐 **Criptografia** → o hash do bloco muda imediatamente
2. 🌐 **Descentralização** → os outros milhares de nós ainda têm a versão original
3. 🤝 **Consenso** → a rede rejeita a versão adulterada e mantém a original

> Resultado: o atacante precisaria **refazer todos os blocos seguintes** e **convencer 51% da rede**. Custo astronômico.

O Custo de um Ataque (51%)

Para reescrever a história, seria necessário:

- **Energia** – no PoW, refazer todo o trabalho computacional
- **Hardware** – mais poder de mineração que o resto da rede somada
- **Tempo** – refazer blocos mais rápido que a rede cria novos
- **Dinheiro** – custo inviável na maioria das redes grandes
- **Incentivo** – ainda assim, a rede poderia rejeitar a cadeia

Não é impossível – é economicamente inviável.

Nuances Importantes

- **Imutabilidade não é absoluta** — é probabilística
- **Blocos recentes** são mais fáceis de reverter (poucas confirmações)
- **Redes permissionadas** (Hyperledger) têm imutabilidade mais fraca
- **Forks** podem reescrever história se a maioria concordar
- **LGPD vs. imutabilidade** — tensão real e não resolvida

> Blockchain é **tamper-evident** (evidencia adulteração) mais do que **tamper-proof** (à prova de adulteração).

Para ir além

- 📖 **"Mastering Bitcoin"** — Andreas Antonopoulos (cap. 7 e 9)
- 📖 **"Blockchain Basics"** — Daniel Drescher
- 📹 **Anders Brownworth** — *Blockchain Demo* (visual interativo)
- 📹 **3Blue1Brown** — vídeo sobre hashes e SHA-256
- 🧪 **Etherscan** e **Blockchain.com** — exploradores reais

Perguntas?



Criptografia

Hashes encadeados



Descentralização

Cópias em milhares de nós



Consenso

Regras aceitas por todos

Pilar 1

Criptografia

Como funciona

- Cada bloco tem um **hash único**
- Cada bloco guarda o **hash do anterior**
- Alterar qualquer dado → hash muda
- O encadeamento de hashes forma uma "corrente inviolável"

Garantias

- **Integridade** – o dado não foi alterado
- **Autenticidade** – assinaturas digitais
- **Não-repúdio** – quem assinou, assinou

Pilar 2

Descentralização

Como funciona

- A blockchain é **copiada em milhares de nós**
- Cada nó valida e guarda a cópia completa
- Não existe um "servidor central"
- Se um nó cai, os outros continuam

Garantias

- 🛡️ **Resistência a censura** – ninguém desliga tudo
- 🔁 **Alta disponibilidade** – sempre há cópias
- 📦 **Confiança distribuída** – não depende de um ator

Pilar 3

Consenso

Como funciona

- Regras definem **o que é um bloco válido**
- Nós "votam" implicitamente ao aceitar/rejeitar blocos
- Só a **cadeia mais longa** (ou mais votada) vence
- Blocos inválidos são ignorados

Algoritmos comuns

- ⛏️ **Proof of Work (PoW)** – Bitcoin
- 💰 **Proof of Stake (PoS)** – Ethereum 2.0
- 🏛️ **PBFT / Raft** – redes permissionadas
- 🗳️ **Delegated PoS** – EOS, Tron

Os Três Pilares em Ação

> Cenário: **um atacante tenta alterar uma transação antiga**.

Os Três Pilares em Ação

1. 🔐 **Criptografia** → o hash do bloco muda imediatamente
2. 🌐 **Descentralização** → os outros milhares de nós ainda têm a versão original
3. 🤝 **Consenso** → a rede rejeita a versão adulterada e mantém a original

> ❌ Resultado: o atacante precisaria **refazer todos os blocos seguintes** e **convencer 51% da rede**. Custo astronômico.

O Custo de um Ataque (51%)

> Para reescrever a história, seria necessário:

- **Energia** — no PoW, refazer todo o trabalho computacional
- **Hardware** — mais poder de mineração que o resto da rede somada
- **Tempo** — refazer blocos mais rápido que a rede cria novos
- **Dinheiro** — custo inviável na maioria das redes grandes
- **Incentivo** — ainda assim, a rede poderia rejeitar a cadeia

Não é impossível — é economicamente inviável.

Conclusões

- **Imutabilidade não é absoluta** — é probabilística
- **Blocos recentes** são mais fáceis de reverter (poucas confirmações)
- **Redes permissionadas** (Hyperledger) têm imutabilidade mais fraca
- **Forks** podem reescrever história se a maioria concordar
- **LGPD vs. imutabilidade** — tensão real e não resolvida

> 🧠 Blockchain é **tamper-evident** (evidencia adulteração) mais do que **tamper-proof** (à prova de adulteração).

Referências

- Merkle Trees e Merkle Roots

José Roberto Bezerra

jbroberto76/minibc

Powered by Slidev



COVER IMAGE BY haikei